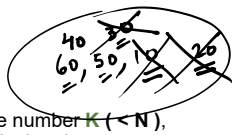
K Largest Elements



Given an array **nums** of **N** integers and a positive number ~~**K** (< **N**)~~, design an algorithm to print the **K** largest elements in the given array.

Note : You can print the K largest element in any order.

Example

Input : nums = [~~60, 50, 10~~, 20, 30, 40], K = 3
Output : { 40, 50, 60 }

1st appr : 1. sort arr : nlogn
2. extract top-k elements : k
nlogn + k

2nd appr 1. transform array into a maxHeap : n
2. extract top-k elements : klogn
n + klogn

3rd appr 1. insert the 1st k-values into a minHeap → klogk
2. scan the remaining elements (n-k) and for each element check if that element is > minHeap.top() ~ if it is then do a minHeap.pop() and track met element → (n-k)(logk + logk) (~logk) } Σ
3. extract all the elements in the heap → klogk

= klogk + nlogk - klogk + klogk
= klogk + nlogk

nlogn + k , n + klogn , klogk + nlogk =

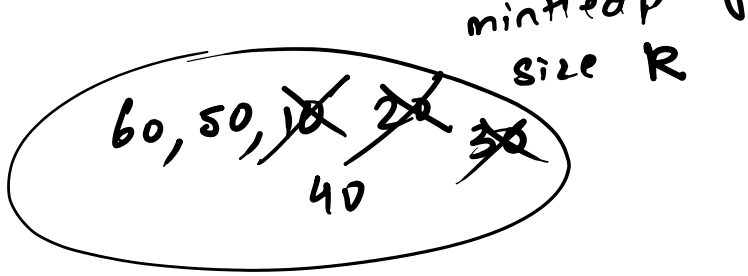
K Largest Elements in a Stream

Given a **running stream** of integers and a positive integer **K**, design an algorithm to print the **K largest elements** in the stream every time you read a **-1** from the stream.

Note : You can print the K largest element in any order.

Example

Input : [60, 50, 10, -1, 20, 30, -1, 40, -1, . . .], K = 3
Output : { 10, 50, 60 }
{ 30, 50, 60 }
{ 40, 50, 60 }
. . .



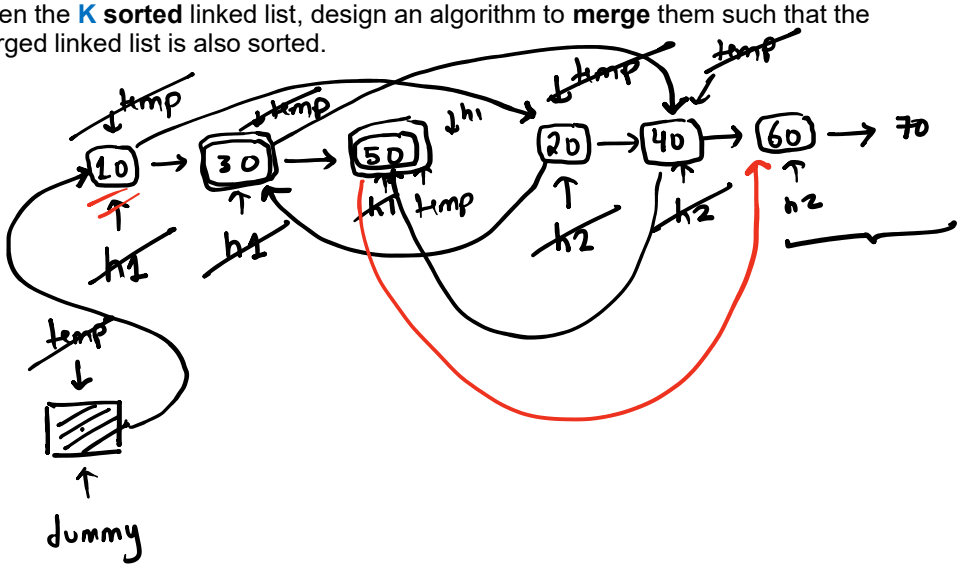
10, 50, 60

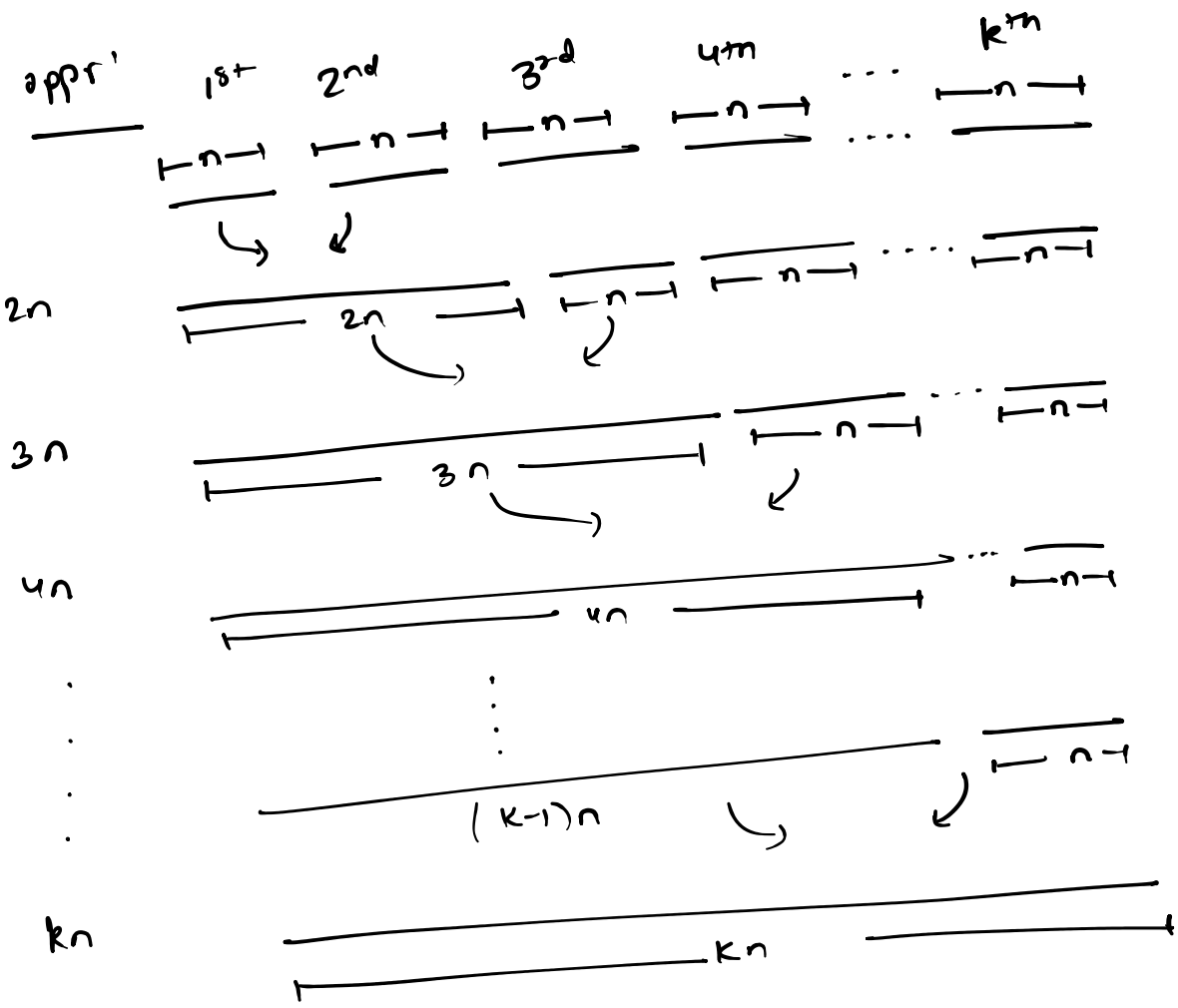
30, 50, 60

40, 50, 60 ....

Merge K-Sorted Linked List

Given the **K** **sorted** linked list, design an algorithm to **merge** them such that the merged linked list is also sorted.





$$= 2n + 3n + 4n + \dots + kn$$

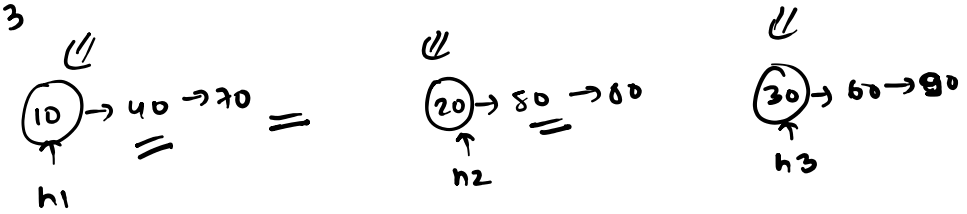
$$(n \cdot 1 + 4 + \dots + k)$$

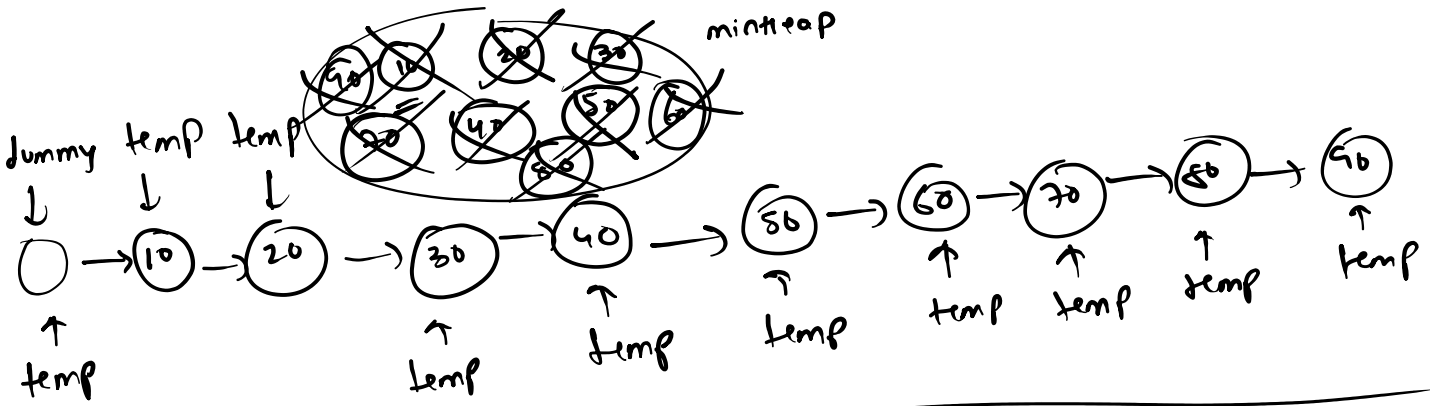
$= 2n + 3n + 4n + \dots$

$= n(\underbrace{2 + 3 + 4 + \dots + k}_{\sim \frac{k(k+1)}{2} \sim k^2})$

$= nk^2$

$k = 3$





$kn(\underbrace{\log k + \log k}_{\sim \log k})$

$= nk\log k$

Merge K-Sorted Arrays

Given the **K** **sorted** arrays, design an algorithm to **merge** them such that the merged array is also sorted.

Example

Input

	0	1	2	3
0	1	3	7	10
1	2	4	5	11
2	0	6	8	9

Output

0	1	2	3	4	5	6	7	8	9	10	11

Reorganize String

Given a string, design an algorithm to check if the characters of the given string can be **reorganized** in a way that **no two characters** with the **same value are adjacent to each other.**

Example

Input : "aab"

Input : "aabca"

Input : "aaab"

Median of a Running Stream

Given a running stream of integers, design an algorithm to print the **median** of the numbers in the stream each time a new integer is added to the stream.

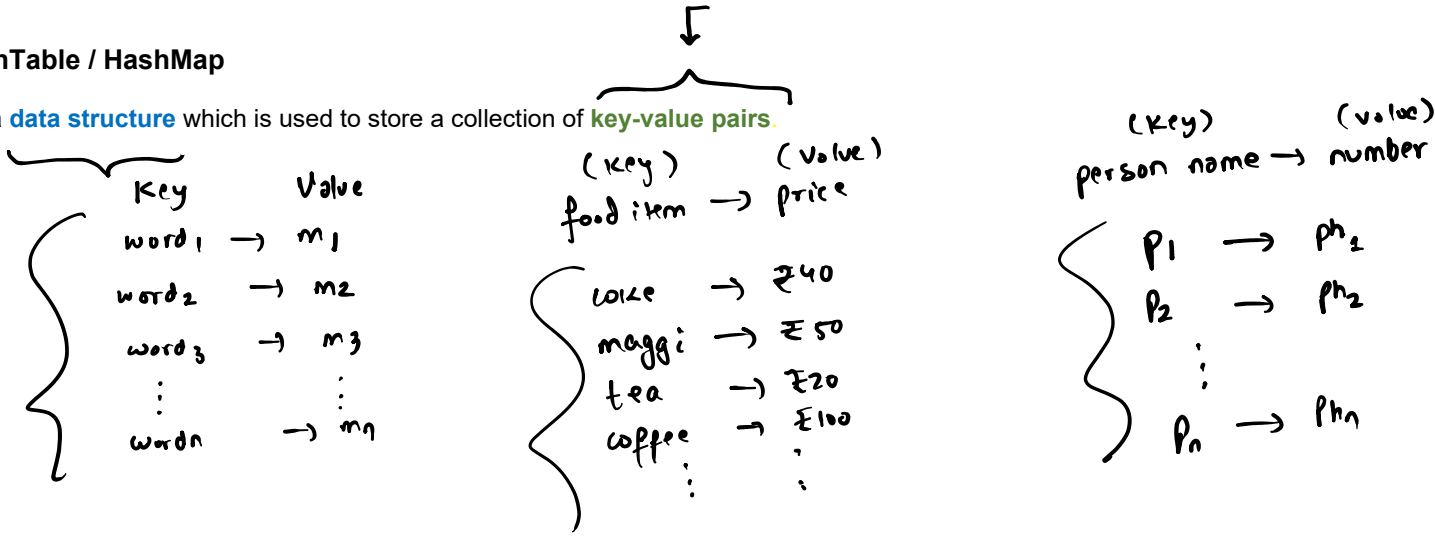
Example

Input

Stream	**4**	**6**	**2**	**5**	**3**	**1**	**7**	**9**	**8**	**. . .**
Median	**4**	**5**	**4**	**4.5**	**4**	**3.5**	**4**	**4.5**	**5**	**. . .**

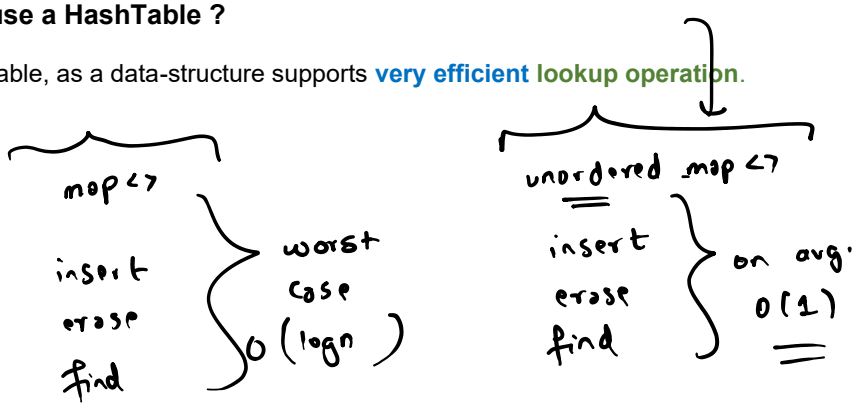
HashTable / HashMap

It is a **data structure** which is used to store a collection of **key-value pairs**.



Why use a HashTable ?

HashTable, as a data-structure supports **very efficient lookup operation**.

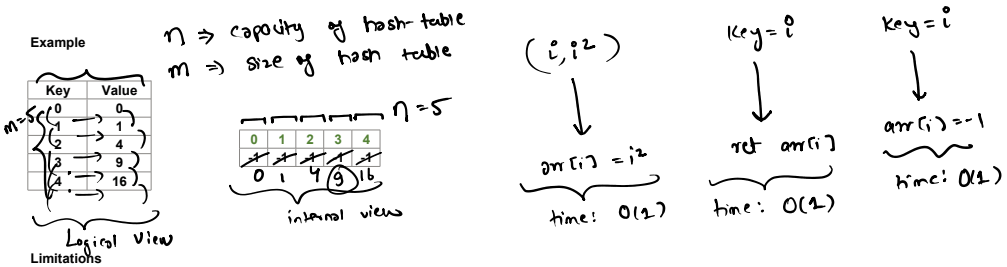


Reason behind Efficiency ?

Data is stored in a HashTable in an **unordered-way.**

HashTable Implementation

Example

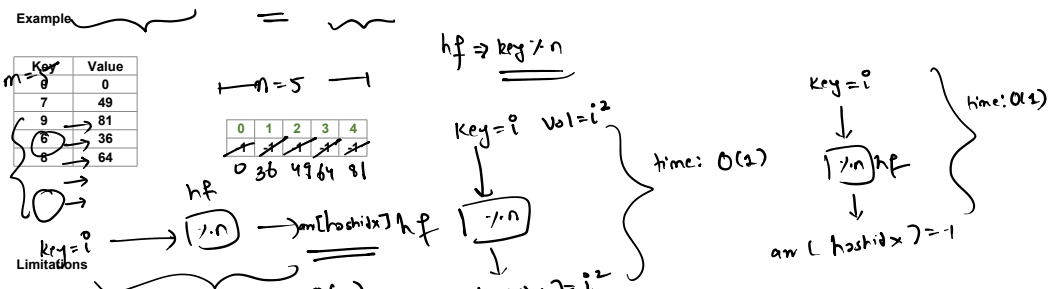


Limitations

- Keys are integer values that **exceed** the valid range of indices.
- Keys are **non-integer values**, e.g. strings

To solve this problem, we use a **hash function** - a mathematic function that transforms the keys into a valid index inside the hash table a.k.a hash index.

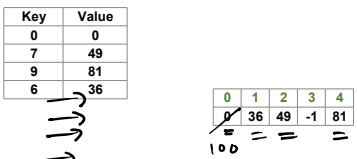
Example



Limitations

- Multiple (key, value) pairs can be mapped to the same index - **collision**

Example

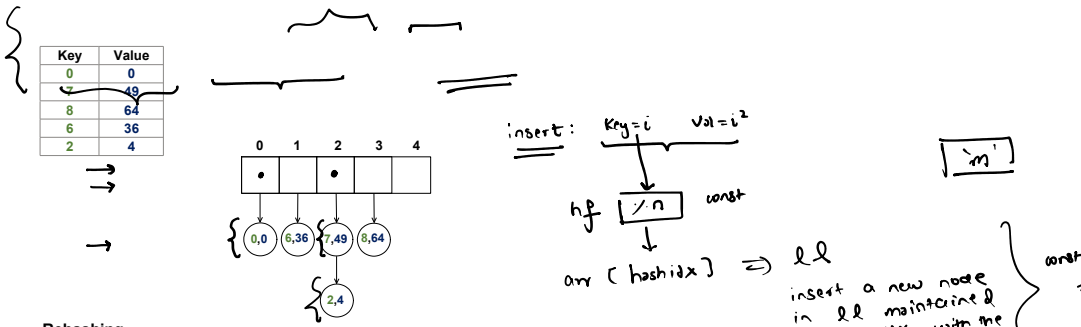


Handling Collision

- **Closed Hashing or Open Addressing**
  - Linear Probing
  - Quadratic Probing
  - Double Hashing

- **Open Hashing or Chaining**

The idea behind **chaining** is to maintain a **linked list** at each index (**or bucket**) of the hash table to store **all** the (key, value) pairs that have been **mapped** to that index.

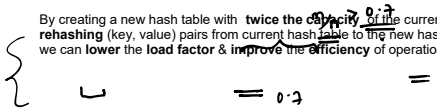


Rehashing

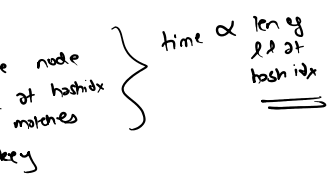
The idea behind rehashing is to **double the capacity** of hash table once the **load factor M/N** exceeds a certain **threshold**.

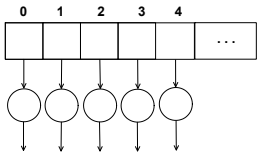
How ?

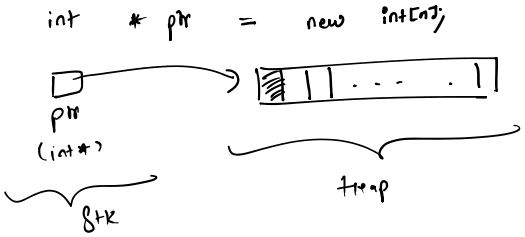
By creating a new hash table with **twice the capacity** of the current hash table and **rehashing** (key, value) pairs from current hash table to the new hash table. This way we can **lower** the **load factor** & **improve** the **efficiency** of operations on hash table.

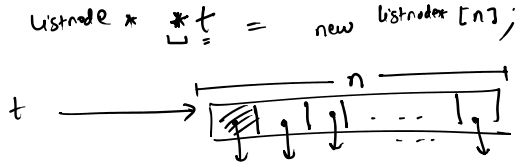


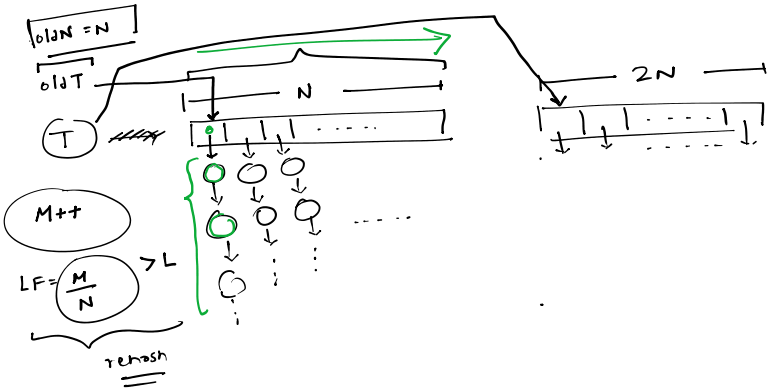
We will internally represent the hash table using a **dynamic array** of **singly linked lists** such each **node of the linked list** present at the i^{th} (0<=i<=N-1) index of the hash table contains a **(key, value)** pair that was hashed/mapped to the i^{th} index of the hash table.


node
at hashidx
matches
key
time ∝ len of ll at hash idx


0
1
2
3
4
...


int * ptr = new int[n];
ptr
(int*)
stk
heap


listnode * *t = new listnode*[n];
t
n


oldN = N
oldT
T
N
2N
M++
LF = M/N
> L
rehash